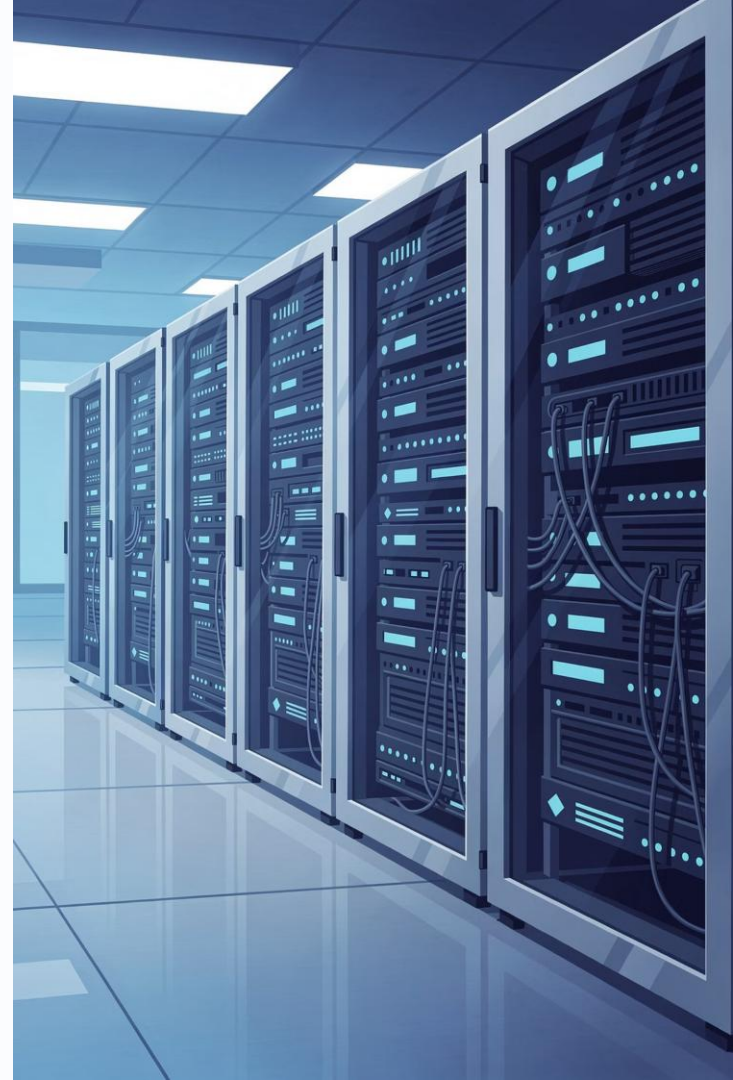


Introducción a la Programación II

Arquitecturas de Despliegue de Software



Agenda y Objetivos de la Sesión

Temas de la Sesión

01

Despliegue de software

Concepto e importancia en el ciclo de vida

02

UML de Componentes y Despliegue

Definición, elementos y ejemplos reales

03

Arquitecturas de Despliegue

Stand Alone, Cliente-Servidor, N-Capas, Cloud

04

Caso práctico integrador

Sistema académico universitario

05

Evaluación y cierre

Preguntas de repaso y reflexión final

Competencias a Desarrollar

Modelar

Diagramas UML de componentes y despliegue

Comparar

Arquitecturas de despliegue según contexto

Seleccionar

La arquitectura más adecuada para una organización



Resultado esperado: Al finalizar, podrás diseñar, modelar y justificar una arquitectura de despliegue para un sistema real.

¿Cómo Llega una App al Usuario Final?

Desde el código del desarrollador hasta la pantalla del usuario, el software recorre un camino complejo. Las decisiones de despliegue impactan directamente en la experiencia, la seguridad y la escalabilidad del sistema.



Netflix

Millones de streams simultáneos gracias a una arquitectura cloud distribuida con microservicios.



Spotify

Recomendaciones personalizadas procesadas en servidores remotos, no en tu dispositivo.



Amazon

Infraestructura de N-capas que separa catálogo, pagos, logística y usuarios.



Sistemas Universitarios

Portales académicos que gestionan matrículas, calificaciones y comunicación institucional.

Introducción al Despliegue de Software

¿Qué es el Despliegue de Software?

El **despliegue** (o *deployment*) es el conjunto de actividades que hacen que un sistema de software esté disponible para su uso. Incluye la instalación, configuración, prueba y puesta en marcha de los componentes en el entorno de producción.

El despliegue es el puente entre el código fuente y el valor real que percibe el usuario final.

Importancia en el Ciclo de Vida del Software



Planificación

Definición de entornos: desarrollo, pruebas, producción.



Integración continua

Automatización del proceso de build y despliegue.



Monitoreo

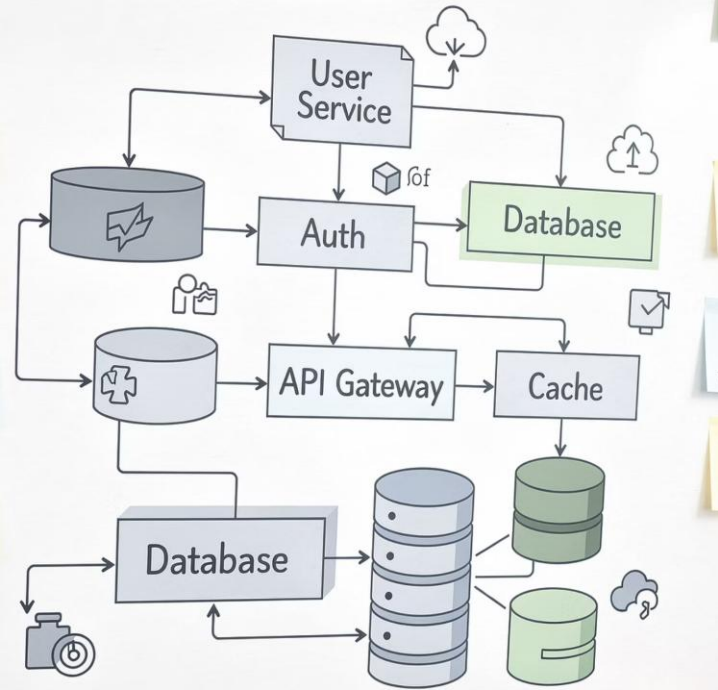
Seguimiento del rendimiento y disponibilidad post-despliegue.



Mantenimiento

Actualizaciones, parches y evolución del sistema en producción.

Arruit. Service



UNIDAD 1 · UML

UML como Herramienta de Modelado

¿Qué es UML?

El **Lenguaje Unificado de Modelado** (UML, *Unified Modeling Language*) es un estándar internacional (ISO 19501) para visualizar, especificar, construir y documentar sistemas de software. No es un lenguaje de programación: es un lenguaje gráfico de comunicación entre equipos técnicos.

UML y la Arquitectura de Despliegue

Diagramas Estructurales

Muestran la organización estática del sistema: clases, componentes, despliegue.

Diagramas de Comportamiento

Describen cómo interactúan los elementos en tiempo de ejecución: secuencia, casos de uso.

❗ Para el despliegue de software usamos principalmente los **diagramas de componentes** y **diagramas de despliegue**.

Diagramas UML de Componentes

Definición y Objetivo

Un **diagrama de componentes** describe la organización y las dependencias entre los módulos físicos del sistema (componentes). Su objetivo es mostrar la arquitectura interna del software: cómo se divide en partes reutilizables e intercambiables.

☐ Responde a la pregunta: **¿De qué partes está hecho el sistema?**

Elementos Principales

Componente

Módulo ejecutable o desplegable con interfaz definida. Representado como un rectángulo con el ícono de componente (dos rectángulos pequeños).

Interfaz (Interface)

Define los servicios que un componente provee (*provided interface*) o requiere (*required interface*).

Dependencia

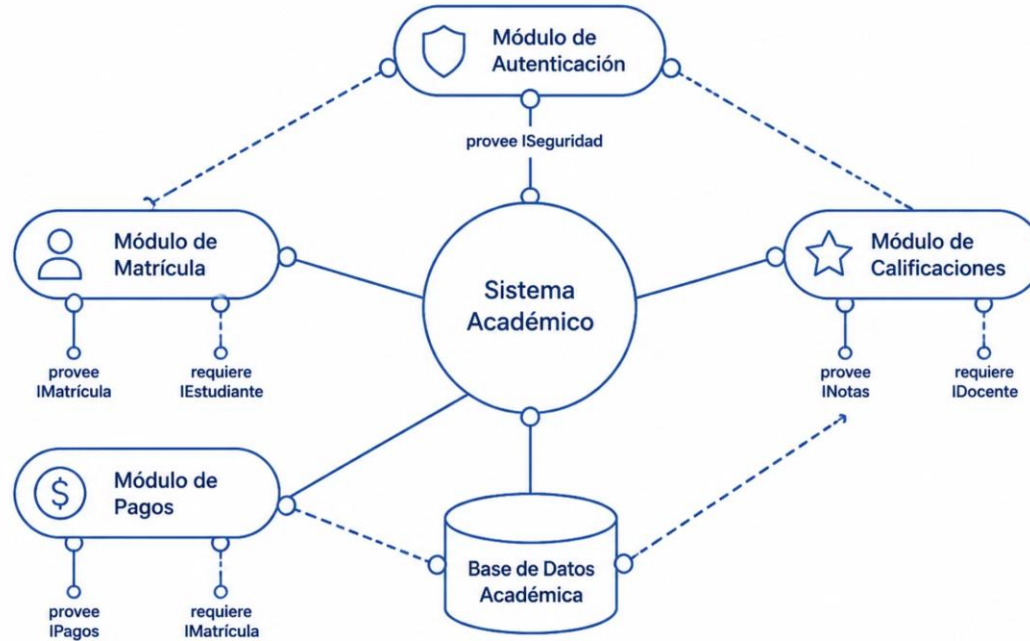
Relación de uso entre componentes, representada con una flecha discontinua.

Subsistema

Agrupación lógica de componentes relacionados dentro de un paquete.

Ejemplo: Sistema Académico Universitario

El siguiente diagrama de componentes muestra cómo se estructura internamente un sistema de gestión académica universitaria, identificando sus módulos principales y sus dependencias.



Lectura del diagrama: Cada módulo es un componente independiente. Las flechas indican dependencias: un componente necesita los servicios de otro para funcionar correctamente.



Diagramas UML de Despliegue

Definición

Un **diagrama de despliegue** modela la arquitectura física del sistema: cómo los artefactos de software se distribuyen y ejecutan sobre los nodos de hardware. Muestra la infraestructura real donde vive el sistema.

❏ Responde a: **¿En qué máquinas y cómo se conectan los componentes?**

Elementos Principales



Nodo (Node)

Recurso físico o virtual que ejecuta software: servidor, dispositivo, VM, contenedor.



Artefacto (Artifact)

Elemento físico del software: archivo .jar, .war, .exe, imagen Docker, script SQL.

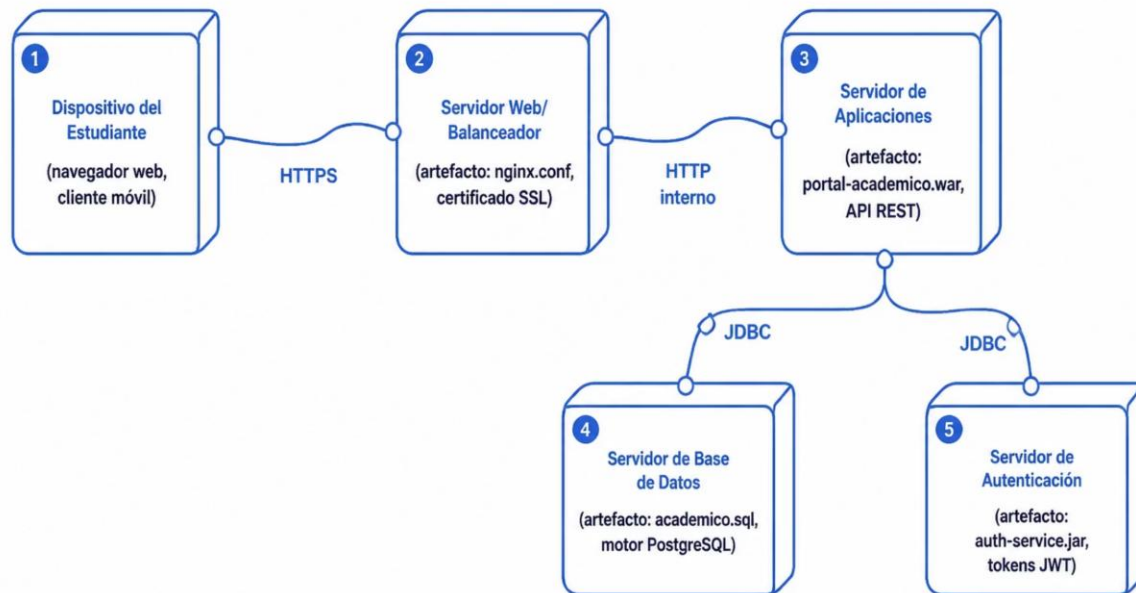


Conexión / Enlace

Canal de comunicación entre nodos: red LAN, internet, protocolo HTTP, TCP/IP.

Ejemplo: Aplicación Web Universitaria

El siguiente diagrama de despliegue representa la infraestructura física de un portal académico universitario, mostrando cómo los artefactos de software se distribuyen en los nodos del sistema.



Clave: Cada nodo representa un servidor físico o virtual. Los artefactos son los archivos reales que se instalan en cada nodo. Las conexiones muestran los protocolos de comunicación.

Componentes vs. Despliegue: ¿Cuándo Usar Cada Uno?

Criterio	Diagrama de Componentes	Diagrama de Despliegue
¿Qué modela?	Estructura lógica del software y sus módulos	Infraestructura física y distribución del sistema
Elementos clave	Componentes, interfaces, dependencias, paquetes	Nodos, artefactos, conexiones, protocolos
Perspectiva	Vista lógica / arquitectura de software	Vista física / arquitectura de infraestructura
¿Cuándo usarlo?	Diseñar módulos, documentar APIs internas, refactorizar	Planificar infraestructura, DevOps, migraciones a cloud
Audiencia	Arquitectos de software, desarrolladores senior	Administradores de sistemas, DevOps, clientes
Ejemplo real	Módulos de un ERP universitario	Servidores y red de un portal de matrículas

  Ambos diagramas son complementarios: primero modelas los componentes lógicos, luego decides cómo desplegarlos físicamente.



Actividad: Analiza una App de Compras en Línea

Escenario

Una tienda en línea como **Mercado Libre** permite buscar productos, agregar al carrito, pagar y rastrear envíos. El sistema tiene millones de usuarios simultáneos.

Tu misión:

1 Identifica los componentes

Lista mínimo 5 módulos del sistema (ej. catálogo, carrito, pagos...)

2 Dibuja el diagrama de componentes

Usa <https://online.visual-paradigm.com/> o papel, muestra interfaces y dependencias

3 Identifica los nodos de despliegue

¿En qué servidores viviría cada componente?

Preguntas Guía para la Reflexión

😬 ¿El módulo de pagos debería estar en el mismo servidor que el catálogo?

😬 ¿Qué pasa si el servidor de base de datos falla?

😬 ¿Cómo se comunica el cliente móvil con el servidor?

Arquitecturas de Despliegue de Software

Una **arquitectura de despliegue** define cómo se organiza y distribuye el software en el entorno de producción. Elegir la arquitectura correcta es una decisión crítica que afecta la escalabilidad, seguridad, costos y mantenimiento del sistema a largo plazo.



Stand Alone

Todo en un solo equipo local



Cliente-Servidor

Separación entre cliente y servidor centralizado



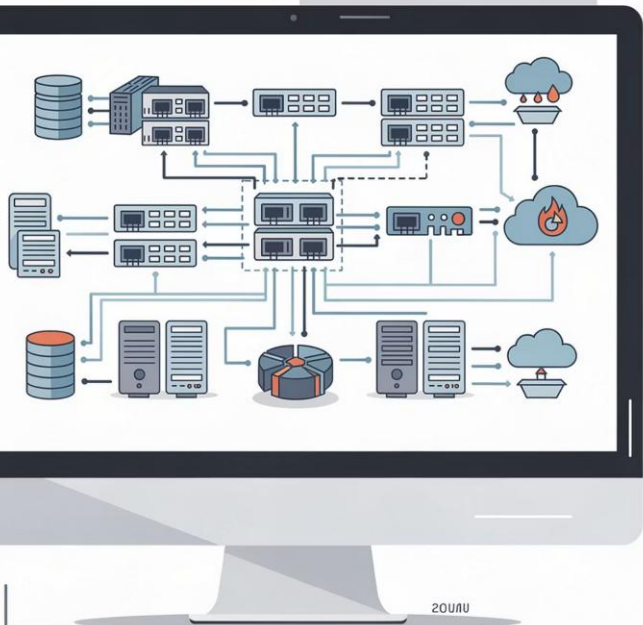
N-Capas

Separación en capas lógicas y físicas



Cloud

Infraestructura distribuida en la nube



Arquitectura Stand Alone

¿Qué es?

En la arquitectura **Stand Alone** (o monolítica local), toda la aplicación —interfaz, lógica de negocio y datos— reside y se ejecuta en **un único equipo** sin depender de red externa. Es el modelo más simple de despliegue.

Características

→ Sin dependencia de red

Funciona completamente offline en la máquina local.

→ Instalación local

El software se instala directamente en el equipo del usuario.

→ Recursos limitados

Depende del hardware del equipo donde está instalado.



✓ Ventajas

Simple, sin latencia de red, bajo costo inicial, fácil de instalar.

✗ Desventajas

Sin acceso remoto, difícil de actualizar en múltiples equipos, sin escalabilidad.

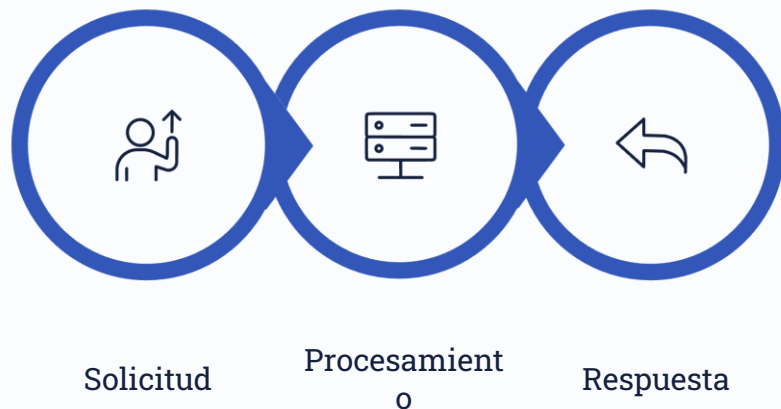


★ **Ejemplo real:** Software de contabilidad instalado localmente (ej. versiones antiguas de Mónica, QuickBooks Desktop).

Arquitectura Cliente-Servidor

¿Cómo Funciona?

El modelo **cliente-servidor** divide el sistema en dos roles: el **cliente** (quien solicita servicios) y el **servidor** (quien los provee). La comunicación ocurre a través de una red mediante protocolos estándar como HTTP, FTP o TCP/IP.



Componentes del Modelo

Cliente (Thin/Fat)

Puede ser un navegador web, app móvil o cliente de escritorio. Envía solicitudes y presenta resultados.

Servidor

Centraliza la lógica de negocio y gestiona recursos compartidos: datos, archivos, autenticación.

Red / Protocolo

Canal de comunicación entre cliente y servidor. Define las reglas del intercambio de datos.



Caso real: Sistema de correo electrónico corporativo (Outlook + Exchange Server) o portales web universitarios clásicos.

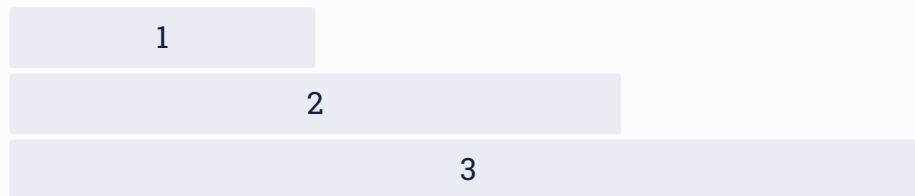
Arquitectura de N-Capas

¿Qué es?

La arquitectura de **N-Capas** (N-Tier) separa el sistema en capas lógicas y/o físicas independientes, donde cada capa tiene una responsabilidad específica y solo se comunica con las capas adyacentes. Permite mayor modularidad y mantenibilidad.

 La arquitectura de 3 capas es la más común: **Presentación** → **Lógica** → **Datos**.

Capas Principales y Beneficios



1 Capa de Datos

Base de datos, repositorios, acceso a datos persistentes.

2 Capa de Lógica de Negocio

Reglas de negocio, validaciones, servicios, APIs.

3 Capa de Presentación

Interfaz de usuario: web, móvil, escritorio.



Ejemplo: Sistema ERP empresarial (SAP, Oracle EBS) o un portal bancario en línea.

Arquitectura Cloud



Conceptos Fundamentales

La **arquitectura Cloud** traslada el software y la infraestructura a proveedores de servicios en internet. Los recursos (cómputo, almacenamiento, red) se consumen bajo demanda y se pagan por uso, eliminando la necesidad de infraestructura física propia.

Modelos de Servicio

IaaS

Infraestructura como Servicio: máquinas virtuales, redes. (AWS EC2, Google Compute Engine)

PaaS

Plataforma como Servicio: entorno de desarrollo y despliegue. (Heroku, Google App Engine)

SaaS

Software como Servicio: aplicaciones completas. (Google Workspace, Office 365, Moodle Cloud)

🕒 **Ejemplos actuales:** Netflix (AWS), Spotify (Google Cloud), Zoom (Oracle Cloud), sistemas académicos LMS en la nube.

Comparación de Arquitecturas de Despliegue

Criterio	Stand Alone	Cliente-Servidor	N-Capas	Cloud
Escalabilidad	✗ Ninguna	⚠ Limitada	✓ Alta	✓ ✓ Elástica
Seguridad	✓ Alta (aislada)	⚠ Media	✓ Alta	✓ Depende del proveedor
Costos iniciales	✓ Muy bajos	⚠ Medios	✗ Altos	✓ Bajos (pago por uso)
Mantenimiento	⚠ Manual por equipo	✓ Centralizado	⚠ Complejo	✓ Delegado al proveedor
Acceso remoto	✗ No	✓ Sí (LAN/WAN)	✓ Sí	✓ Global
Casos de uso	Software local simple	Sistemas departamentales	ERP, banca, gobierno	SaaS, startups, educación

⚠ ⚠ No existe una arquitectura universalmente superior. La elección depende del contexto, presupuesto y requisitos del sistema.

Caso: Modernización del Sistema Académico Universitario



Escenario

Una universidad regional con 15,000 estudiantes desea modernizar su sistema académico y permitir acceso remoto para estudiantes y docentes desde cualquier lugar del mundo.

Arquitectura Actual: Stand Alone + Cliente-Servidor Básico

Problemas detectados

- El sistema colapsa en períodos de matrícula
- No permite acceso remoto para estudiantes virtuales
- Actualizaciones requieren visitas físicas a cada equipo
- Los datos no tienen respaldo automático en la nube



Arquitectura Recomendada: Cloud + N-Capas

01

Capa de Presentación (SaaS)

Portal web y app móvil accesibles desde cualquier dispositivo.

02

Capa de Lógica (PaaS)

APIs REST en contenedores Docker gestionados por Kubernetes.

03

Capa de Datos (IaaS/DBaaS)

Base de datos PostgreSQL gestionada en AWS RDS con réplicas.

04

Seguridad y Autenticación

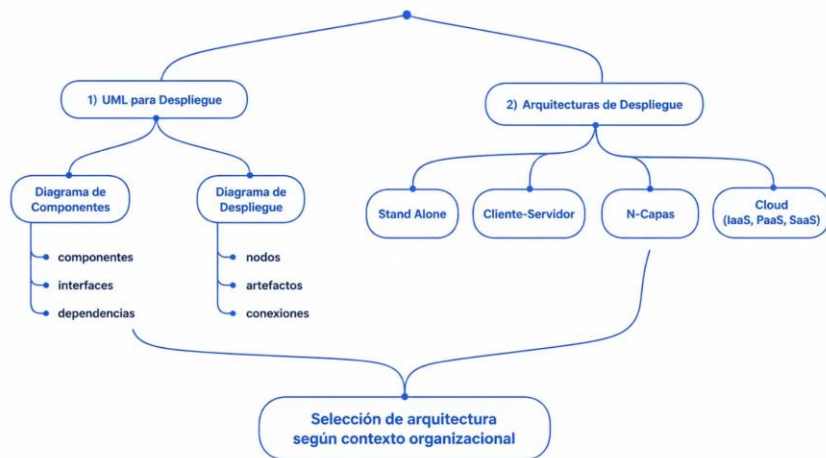
OAuth 2.0, HTTPS, WAF (Web Application Firewall) en la nube.



✓ **Justificación técnica:** La combinación Cloud + N-Capas garantiza escalabilidad automática, disponibilidad 24/7, acceso global y reducción de costos operativos en un 40% vs. infraestructura propia.

Ideas Clave y Buenas Prácticas

Mapa de Conceptos de la Sesión



✓ Buenas Prácticas de Despliegue

→ Documenta siempre

Un diagrama de despliegue actualizado evita errores en producción.

→ Separa entornos

Nunca mezcles desarrollo, pruebas y producción en el mismo servidor.

→ Planifica la escalabilidad

Diseña para el crecimiento futuro, no solo para el presente.

→ Automatiza el despliegue

Usa CI/CD pipelines para reducir errores humanos.

→ Prioriza la seguridad

La seguridad no es una característica adicional: es parte del diseño base.